

MATH518 Coursework

Group 10

2026-05-08

Task 1.1

The dataset was imported into R and the variable `activity` was converted into a factor called `activity_f`, with `isolated` set as the reference level. The frequency table for `activity_f` is shown below.

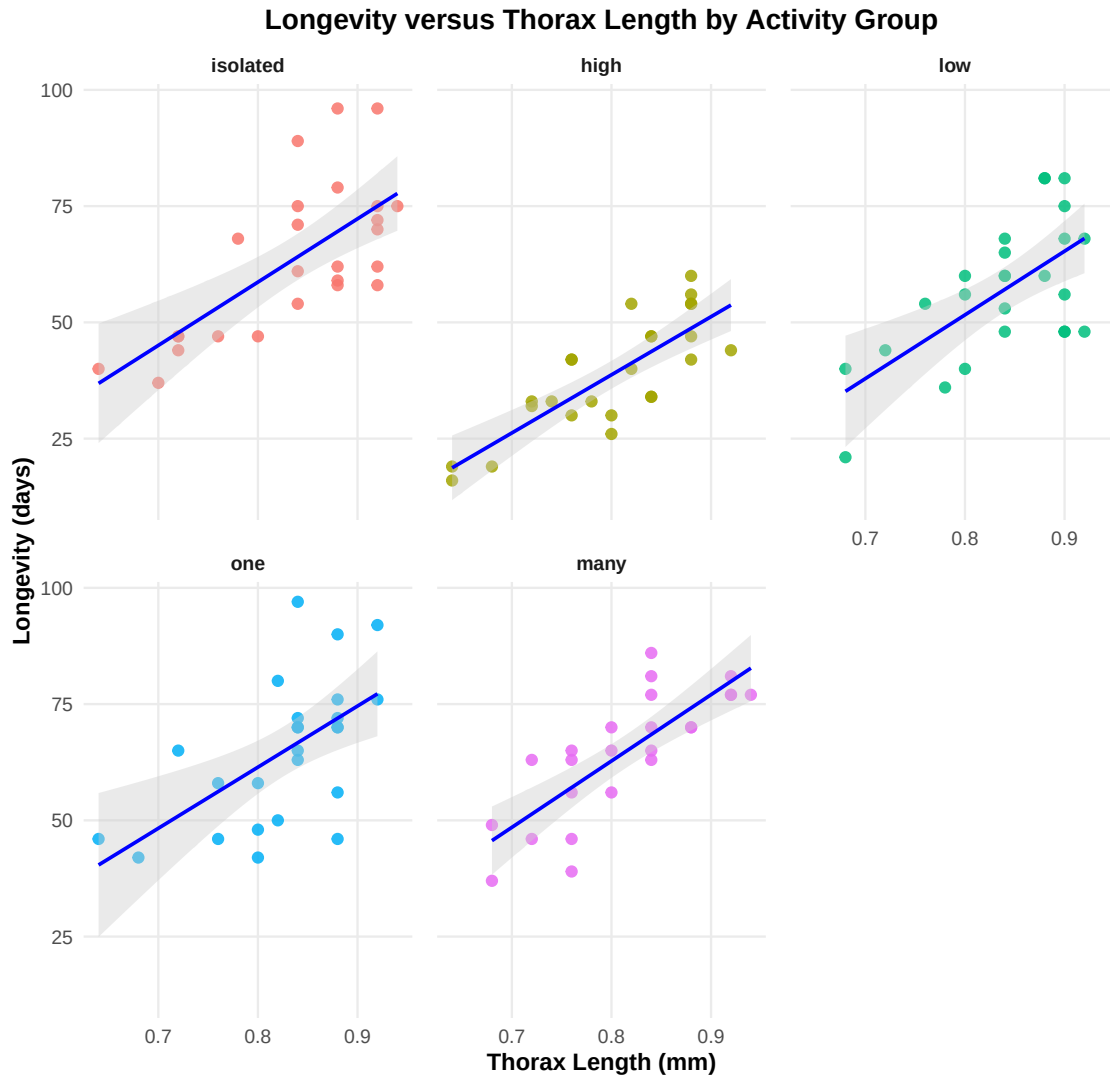
```
fruitflies <- read_csv("fruitflies.csv")

fruitflies <- fruitflies %>%
  mutate(
    activity_f = factor(
      activity,
      levels = c("isolated", "high", "low", "one", "many")
    )
  )
```

```
## # A tibble: 5 x 2
##   activity_f      n
##   <fct>        <int>
## 1 isolated      25
## 2 high          25
## 3 low           25
## 4 one           25
## 5 many          24
```

The frequency table shows that the groups are almost perfectly balanced. The isolated, high, low and one groups each contain 25 fruitflies, while the many group contains 24. This balanced design is helpful for fair comparison between activity groups in later modelling.

Task 1.2



The plot shows a positive association between thorax length and longevity within each activity group, suggesting that larger fruitflies generally tend to live longer. The fitted lines also differ across groups in both level and slope, indicating that activity group may affect expected longevity and may also modify the relationship between thorax length and longevity. This supports the use of an interaction model in the next part of the analysis.

Task 1.3

A linear regression model with an interaction between thorax length and activity group was fitted to assess whether the relationship between thorax length and longevity differs across activity groups.

```
m1 <- lm(longevity ~ thorax * activity_f, data = fruitflies)
summary(m1)
```

```
##
```

```
## Call:
## lm(formula = longevity ~ thorax * activity_f, data = fruitflies)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -25.9509  -6.7296  -0.9103   6.1854  30.3071
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    -50.2420     21.8012  -2.305   0.023 *
## thorax         136.1268     25.9517   5.245 7.27e-07 ***
## activity_fhigh -11.0380     31.2866  -0.353   0.725
## activity_flow  -7.7501     33.9690  -0.228   0.820
## activity_fone   6.5172     33.8708   0.192   0.848
## activity_fmany  -1.1394     32.5298  -0.035   0.972
## thorax:activity_fhigh -11.1268     38.1200  -0.292   0.771
## thorax:activity_flow  0.8743     40.4253   0.022   0.983
## thorax:activity_fone -4.6771     40.6518  -0.115   0.909
## thorax:activity_fmany  6.5478     39.3600   0.166   0.868
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 10.71 on 114 degrees of freedom
## Multiple R-squared:  0.6534, Adjusted R-squared:  0.626
## F-statistic: 23.88 on 9 and 114 DF,  p-value: < 2.2e-16
```

The interaction model explains a substantial proportion of the variability in longevity, with an adjusted R^2 of 0.626. The overall regression is highly significant ($p < 2.2 \times 10^{-16}$), indicating that thorax length and activity group together are important predictors of lifespan.

For the reference group (*isolated*), thorax length is positively associated with longevity ($\hat{\beta} = 136.13$, $p < 0.001$), suggesting that larger fruitflies tend to live longer. However, there is little statistical evidence that either baseline longevity or the effect of thorax length differs between activity groups, since none of the interaction terms are significant at the 5% level.

Although the fitted lines in Task 1.2 showed some visual separation, the interaction model suggests these apparent differences are consistent with sampling variability rather than strong group-specific slope effects.

Task 1.4

To assess whether separate group-specific slopes are required, the full interaction model was compared with a reduced model containing different intercepts but a common slope across groups.

```
m2 <- lm(longevity ~ thorax + activity_f, data = fruitflies)
anova(m2, m1)
```

```
## Analysis of Variance Table
##
## Model 1: longevity ~ thorax + activity_f
## Model 2: longevity ~ thorax * activity_f
##   Res.Df  RSS Df Sum of Sq    F Pr(>F)
## 1     118 13107
## 2     114 13083   4    24.314 0.053 0.9947
```

```
## Single term deletions
##
## Model:
## longevity ~ thorax * activity_f
##
```

	Df	Sum of Sq	RSS	AIC	F value	Pr(>F)
## <none>			13083	597.69		
## thorax:activity_f	4	24.314	13107	589.92	0.053	0.9947

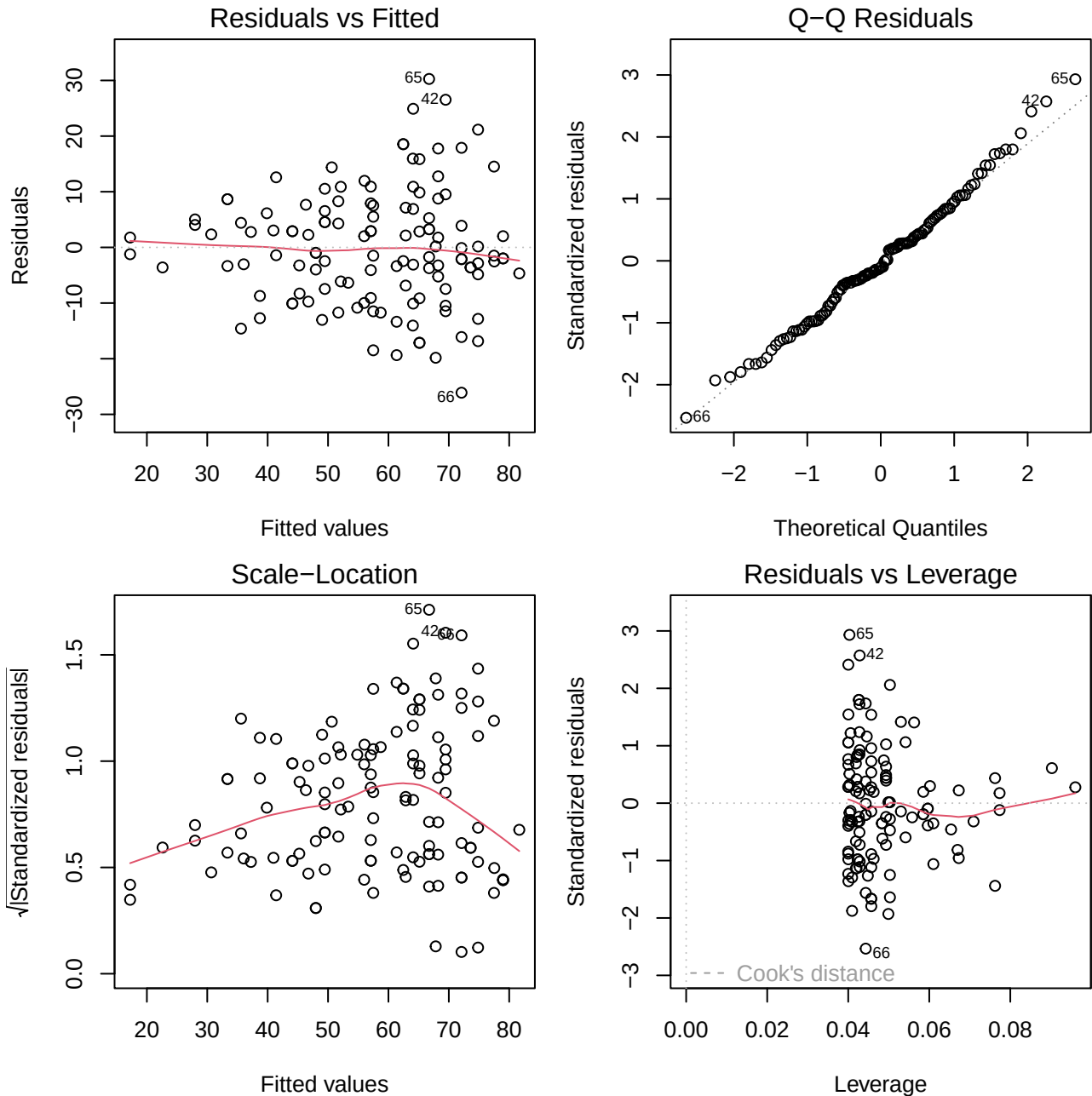
The nested model comparison gives $F = 0.053$ with $p = 0.9947$. Therefore, there is no statistical evidence that the relationship between thorax length and longevity differs across activity groups. In other words, the interaction terms do not provide a meaningful improvement in model fit.

Consequently, model (2), which allows different intercepts but assumes a common slope across groups, is preferred. Since it achieves a comparable fit using fewer parameters, it provides a more parsimonious description of the data.

Applying `drop1(m1, test = "F")` produces the same p-value ($p = 0.9947$) because removing the interaction term from model (1) yields exactly the same reduced model used in the ANOVA comparison. Both procedures therefore test the same null hypothesis under the same least-squares framework.

Task 1.5

The assumptions of model (2) were assessed using standard residual diagnostic plots.



The residual diagnostics suggest that model (2) provides an adequate fit to the data. The Residuals versus Fitted plot shows no strong systematic structure, supporting the assumption of linearity, although there is some indication of slightly increased variability at larger fitted values.

The Normal Q-Q plot lies reasonably close to the reference line, with only minor departures in the tails, suggesting that the normality assumption is broadly satisfied. Similarly, the Scale-Location plot indicates mild heteroscedasticity, but not at a level that appears severe.

The Residuals versus Leverage plot shows no observations with unusually high leverage or influence. Overall, the assumptions underlying the linear regression model appear reasonably well satisfied, with only minor evidence of non-constant variance that is unlikely to materially affect the main conclusions.

Task 1.6

The analysis was repeated using a log-transformed response, where the response variable was defined as $\log(\text{longevity})$. This transformation was considered because the residual analysis for the untransformed model suggested mild non-constant variance.

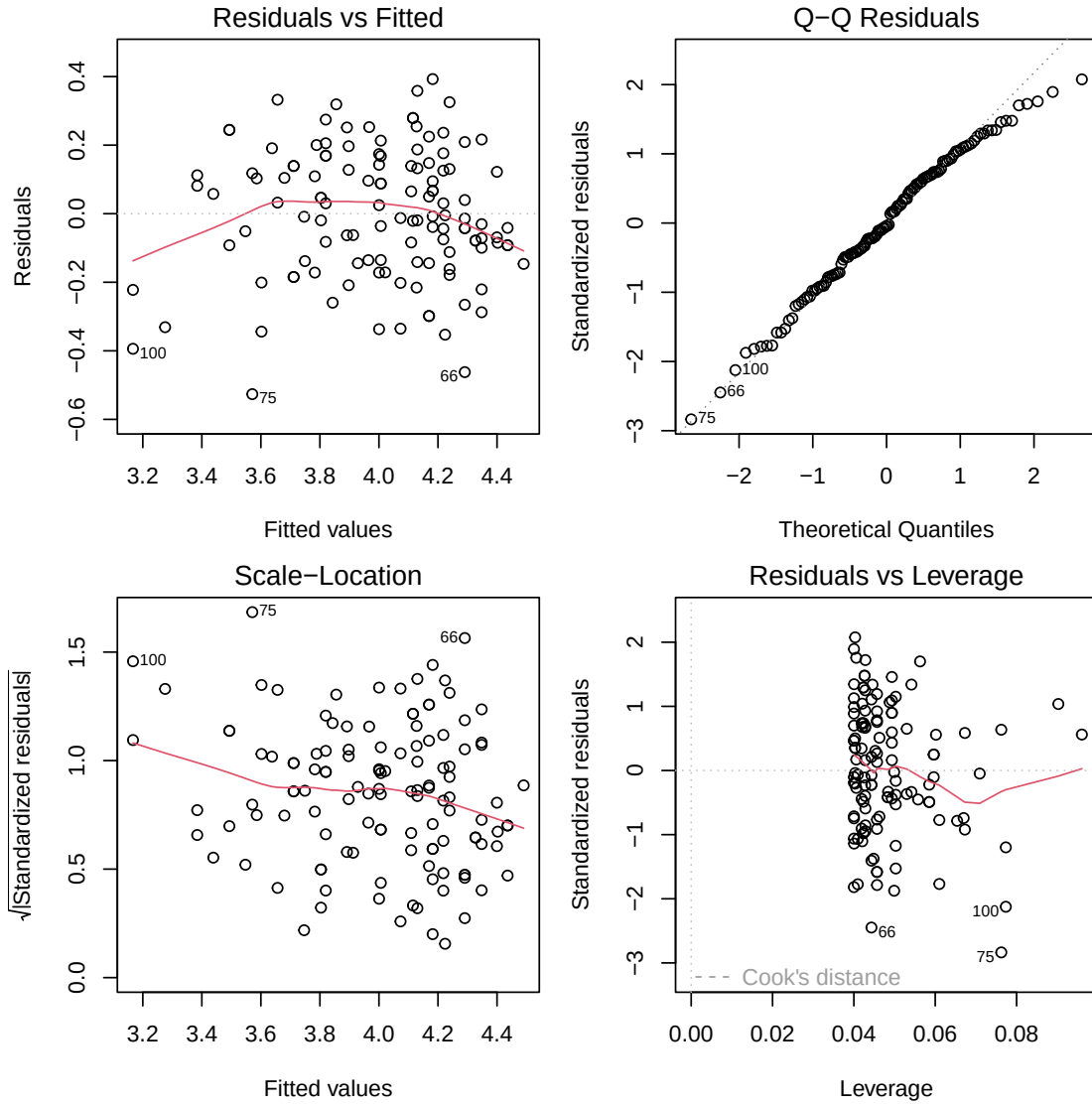
```
## Analysis of Variance Table
##
## Model 1: log_longevity ~ thorax + activity_f
## Model 2: log_longevity ~ thorax * activity_f
##   Res.Df    RSS Df Sum of Sq    F Pr(>F)
## 1     118 4.4012
## 2     114 4.1626  4   0.23857 1.6334 0.1706

## Single term deletions
##
## Model:
## log_longevity ~ thorax * activity_f
##           Df Sum of Sq    RSS      AIC F value Pr(>F)
## <none>                4.1626 -400.87
## thorax:activity_f  4   0.23857 4.4012 -401.96  1.6334 0.1706
```

The nested model comparison gives $F = 1.633$ with $p = 0.171$. Since this p-value is greater than 0.05, there is insufficient evidence that the thorax slope differs between activity groups after log-transforming longevity. The `drop1` output gives the same p-value because deleting the interaction term from the full model produces the same reduced common-slope model used in the ANOVA comparison.

Therefore, the preferred log-transformed model is model (2), which allows different intercepts for each activity group but assumes a common thorax slope:

$$\log(\text{longevity}) \sim \text{thorax} + \text{activity_f}.$$



Residual diagnostics suggest that the log transformation improves adherence to the regression assumptions. In particular, the residual spread appears more stable and approximate normality is reasonably satisfied, while no highly influential observations are evident.

Overall, the log-transformed model gives the same modelling conclusion as before: separate group-specific slopes are not required. The residual diagnostics are broadly acceptable and show no serious violations of the regression assumptions. Since the log transformation addresses the mild non-constant variance seen in the original model, the common-slope model with $\log(\text{longevity})$ is preferred for the subsequent analysis.

Task 1.7

The suggestion to combine activity groups is sensible because the earlier analysis provided little evidence that separate thorax–longevity slopes were required. This indicates that the primary differences between activity groups are captured through differences in intercepts rather than differences in slope. If some categories exhibit similar fitted responses, combining them can simplify the model, reduce the number of parameters, and improve interpretability without a substantial loss of information.

```
# Create coarser grouping structure
fruitflies <- fruitflies %>%
  mutate(
    activity_coarse = fct_collapse(
      activity_f,
      isolated_or_worse = c("isolated", "one", "many"),
      low = "low",
      high = "high"
    ),
    activity_coarse = fct_relevel(
      activity_coarse,
      "isolated_or_worse", "low", "high"
    )
  )

## # A tibble: 3 x 2
##   activity_coarse      n
##   <fct>           <int>
## 1 isolated_or_worse    74
## 2 low                 25
## 3 high                 25

##
## Call:
## lm(formula = log_longevity ~ thorax + activity_coarse, data = fruitflies)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.53214 -0.13509 -0.00245  0.14685  0.39903
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    1.92016    0.19324   9.937  < 2e-16 ***
## thorax         2.68514    0.23267  11.540  < 2e-16 ***
## activity_coarselow -0.16940    0.04488  -3.775  0.00025 ***
## activity_coarsehigh -0.46615    0.04515 -10.324  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1936 on 120 degrees of freedom
## Multiple R-squared:  0.6961, Adjusted R-squared:  0.6885
## F-statistic: 91.61 on 3 and 120 DF, p-value: < 2.2e-16

## Analysis of Variance Table
##
```



```
## Model 1: log_longevity ~ thorax + activity_coarse
## Model 2: log_longevity ~ thorax + activity_f
##   Res.Df    RSS Df Sum of Sq    F Pr(>F)
## 1     120 4.4961
## 2     118 4.4012  2  0.094966 1.2731 0.2838

##           df       AIC
## m_coarse  5 -49.41957
## m_full    7 -48.06672
```

The coarser model allows different mean log-longevity levels across the three broader activity categories while assuming a common thorax slope.

The ANOVA comparison gives $F = 1.273$ with $p = 0.2838$, indicating no statistical evidence that the more detailed five-group model provides a better fit than the simpler three-group model. In addition, the AIC is lower for the coarser model (-49.42) than for the five-group model (-48.07), suggesting that the simpler model achieves a better balance between model fit and complexity.

Therefore, defining the groups using `activity_coarse` is preferable to using `activity_f`. Combining the similar categories `isolated`, `one`, and `many` produces a more parsimonious and interpretable model without any meaningful loss of explanatory power.

Task 1.8

Using the preferred model from Task 1.7, prediction intervals were calculated for fruitflies in the **high** activity group with thorax lengths 0.7 mm and 0.9 mm. Since the model response is $\log(\text{longevity})$, predictions were first obtained on the log scale and then transformed back to days using the exponential function.

```
## # A tibble: 2 x 5
##   thorax activity_coarse point_estimate_days lower_95_PI_days upper_95_PI_days
##   <dbl> <fct>           <dbl>           <dbl>           <dbl>
## 1    0.7 high           28.0            18.9            41.6
## 2    0.9 high           48.0            32.4            71.1
```

For a fruitfly in the **high** activity group with thorax length 0.7 mm, the predicted longevity is approximately 28.0 days, with a 95% prediction interval of 18.9 to 41.6 days. For a thorax length of 0.9 mm, the predicted longevity increases to approximately 48.0 days, with a 95% prediction interval of 32.4 to 71.1 days.

These results suggest that, within the high activity group, larger thorax length is associated with greater expected longevity.

A 95% prediction interval represents the range within which the longevity of a new individual fruitfly with the specified thorax length and activity group is expected to fall, with 95% confidence under the fitted model. The interval is relatively wide because it incorporates both uncertainty in the estimated mean response and natural variability between individual fruitflies.

Task 1.9

The model using `activity_coarse` treats the grouping variable as a factor. R represents this internally using indicator variables. Since `isolated_or_worse` is the reference category, the factor model should be equivalent to a model containing `thorax`, `is_low` and `is_high`.

```
# Create data frame with indicator variables
fruitflies_3 <- fruitflies %>%
  mutate(
    is_isolated_or_worse = ifelse(activity_coarse == "isolated_or_worse", 1, 0),
    is_low = ifelse(activity_coarse == "low", 1, 0),
    is_high = ifelse(activity_coarse == "high", 1, 0)
  )
```

```
##
## Call:
## lm(formula = log_longevity ~ thorax + activity_coarse, data = fruitflies_3)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.53214 -0.13509 -0.00245  0.14685  0.39903
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      1.92016    0.19324   9.937 < 2e-16 ***
## thorax           2.68514    0.23267  11.540 < 2e-16 ***
## activity_coarselow -0.16940    0.04488  -3.775 0.00025 ***
## activity_coarsehigh -0.46615    0.04515 -10.324 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1936 on 120 degrees of freedom
## Multiple R-squared:  0.6961, Adjusted R-squared:  0.6885
## F-statistic: 91.61 on 3 and 120 DF, p-value: < 2.2e-16

##
## Call:
## lm(formula = log_longevity ~ thorax + is_low + is_high, data = fruitflies_3)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.53214 -0.13509 -0.00245  0.14685  0.39903
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      1.92016    0.19324   9.937 < 2e-16 ***
## thorax           2.68514    0.23267  11.540 < 2e-16 ***
## is_low          -0.16940    0.04488  -3.775 0.00025 ***
## is_high         -0.46615    0.04515 -10.324 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1936 on 120 degrees of freedom
## Multiple R-squared:  0.6961, Adjusted R-squared:  0.6885
## F-statistic: 91.61 on 3 and 120 DF, p-value: < 2.2e-16
```

The two models give equivalent inference. The intercept and thorax coefficient are the same in both models, and the coefficients for `activity_coarselow` and `activity_coarsehigh` in the factor model correspond to the coefficients for `is_low` and `is_high` in the indicator-variable model. This confirms that using a factor variable is equivalent to using dummy variables manually, provided the same reference group is used.

```
##              2.5 %    97.5 %
## (Intercept)    1.5375655  2.3027562
## thorax         2.2244663  3.1458113
## activity_coarse_low -0.2582474 -0.0805469
## activity_coarse_high -0.5555408 -0.3767543
```

```
##              2.5 %    97.5 %
## (Intercept)    1.5375655  2.3027562
## thorax         2.2244663  3.1458113
## is_low         -0.2582474 -0.0805469
## is_high        -0.5555408 -0.3767543
```

The confidence intervals also match between the two parameterisations, confirming that the fitted models are equivalent.

```
##
## Call:
## lm(formula = log_longevity ~ thorax + is_isolated_or_worse +
##      is_low + is_high, data = fruitflies_3)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.53214 -0.13509 -0.00245  0.14685  0.39903
##
## Coefficients: (1 not defined because of singularities)
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    1.45401    0.19012   7.648 5.63e-12 ***
## thorax         2.68514    0.23267  11.540 < 2e-16 ***
## is_isolated_or_worse 0.46615    0.04515  10.324 < 2e-16 ***
## is_low         0.29675    0.05544   5.352 4.23e-07 ***
## is_high                NA         NA      NA      NA
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1936 on 120 degrees of freedom
## Multiple R-squared:  0.6961, Adjusted R-squared:  0.6885
## F-statistic: 91.61 on 3 and 120 DF, p-value: < 2.2e-16
```

It is not possible to estimate all three indicator coefficients together with an intercept because every fruitfly belongs to exactly one of the three groups. Therefore,

$$is_isolated_or_worse + is_low + is_high = 1.$$

This creates perfect multicollinearity, also known as the dummy-variable trap. The intercept is already equal to a column of ones, so including all three group indicators adds redundant information. As a result, R automatically removes one coefficient from the model and reports it as not estimable.

In this case, the coefficient for `is_high` cannot be estimated because it is completely determined by the intercept and the other two indicator variables. To fit the model correctly, one group must be treated as the reference category, or the intercept must be removed. The model using `thorax`, `is_low` and `is_high` is therefore the appropriate indicator-variable version of the factor model with `isolated_or_worse` as the reference group.

Task 1.10

A Bayesian version of the preferred log-longevity regression model was fitted using Stan. The response variable is $\log(\text{longevity})$, with thorax length and the indicators `is_low` and `is_high` as explanatory variables. The reference group is therefore `isolated_or_worse`.

```
stan_code_1 <- "
data {
  int<lower=1> n;
  vector[n] y;
  vector[n] thorax;
  vector[n] is_low;
  vector[n] is_high;
}

parameters {
  real beta0;
  real beta1;
  real beta2;
  real beta3;
  real<lower=0> sigma;
}

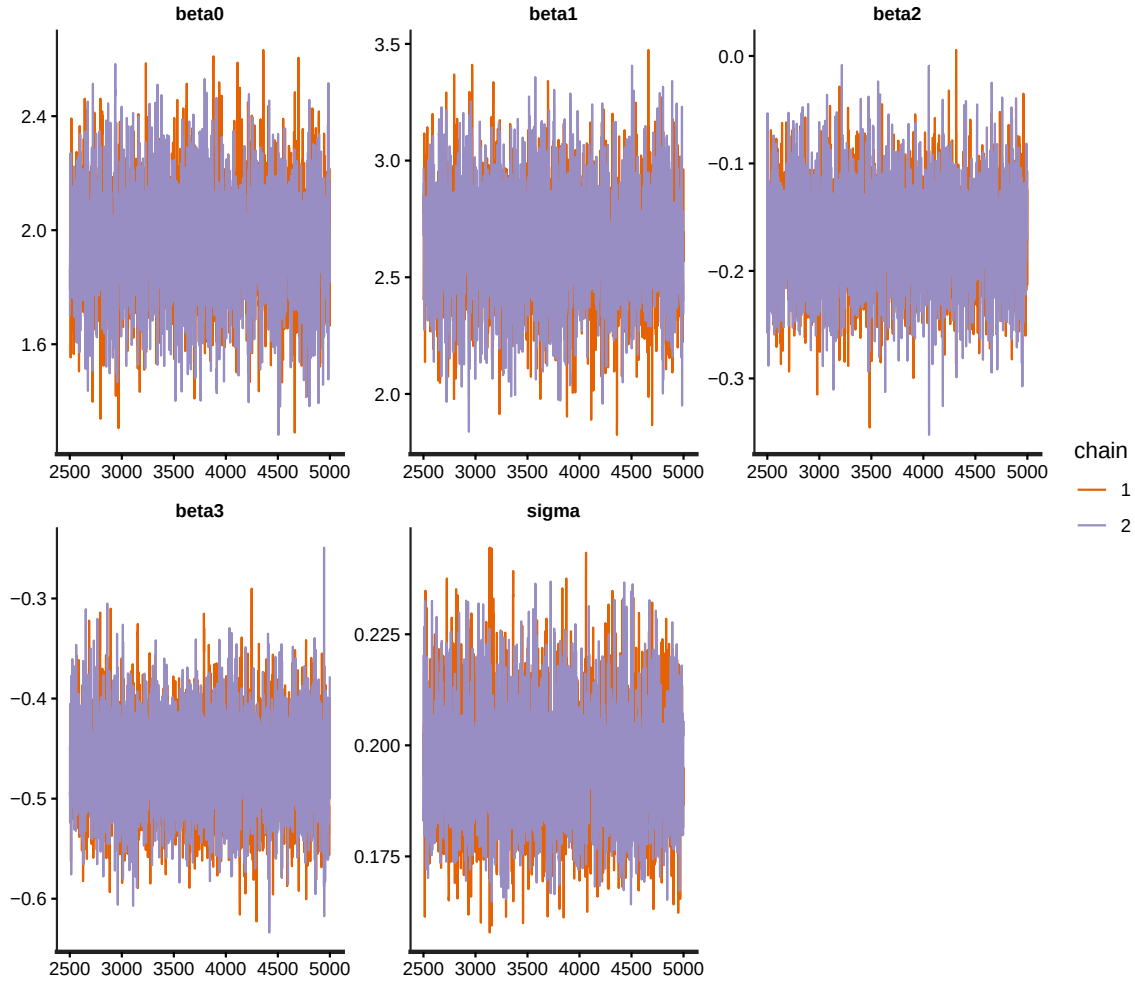
model {
  vector[n] mu;

  beta0 ~ normal(0, 1);
  beta1 ~ normal(0, 1);
  beta2 ~ normal(0, 1);
  beta3 ~ normal(0, 1);
  sigma ~ cauchy(1, 10);

  mu = beta0 + beta1 * thorax + beta2 * is_low + beta3 * is_high;

  y ~ normal(mu, sigma);
}
"
```

```
## Inference for Stan model: anon_model.
## 2 chains, each with iter=5000; warmup=2500; thin=1;
## post-warmup draws per chain=2500, total post-warmup draws=5000.
##
##           mean se_mean   sd  2.5%  50% 97.5% n_eff Rhat
## beta0    1.96         0 0.19   1.57   1.96   2.34  2698    1
## beta1    2.64         0 0.23   2.17   2.64   3.10  2734    1
## beta2   -0.17         0 0.05  -0.26  -0.17  -0.08  4119    1
## beta3   -0.47         0 0.05  -0.56  -0.47  -0.37  3666    1
## sigma    0.20         0 0.01   0.17   0.19   0.22  3520    1
##
## Samples were drawn using NUTS(diag_e) at Fri May  8 13:46:23 2026.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```



The Stan model was run using 2 chains with 5000 iterations each and a warm-up of 2500 iterations, resulting in 5000 post-warmup samples in total.

The traceplots for β_0 , β_1 , β_2 , β_3 , and σ show good mixing between the two chains. The chains overlap closely and fluctuate around stable means with no visible trends or drift, indicating that the Markov chains have reached stationarity.

The numerical diagnostics further support convergence. All $\hat{R}$ values are equal to 1, indicating no evidence of lack of convergence. The effective sample sizes are also large for all parameters (all above 2500), suggesting that the posterior estimates are based on a sufficient number of effectively independent draws.

Overall, there is strong evidence that the MCMC algorithm has converged successfully, and the posterior summaries can be considered reliable for subsequent Bayesian inference.

Task 1.11

Posterior means and 95% credible intervals were extracted from the Bayesian model and compared with the corresponding frequentist 95% confidence intervals from `m_coarse_2`.

```
# Frequentist confidence intervals
confint(m_coarse_2)
```

```
##              2.5 %      97.5 %
## (Intercept)  1.5375655  2.3027562
## thorax       2.2244663  3.1458113
## is_low       -0.2582474 -0.0805469
## is_high      -0.5555408 -0.3767543
```

```
# Frequentist estimate of sigma
summary(m_coarse_2)$sigma
```

```
## [1] 0.1935656
```

```
# Bayesian posterior summaries
print(
  fit_1,
  pars = c("beta0", "beta1", "beta2", "beta3", "sigma"),
  probs = c(0.025, 0.975)
)
```

```
## Inference for Stan model: anon_model.
## 2 chains, each with iter=5000; warmup=2500; thin=1;
## post-warmup draws per chain=2500, total post-warmup draws=5000.
##
##      mean se_mean   sd  2.5% 97.5% n_eff Rhat
## beta0  1.96      0 0.19  1.57  2.34  2698    1
## beta1  2.64      0 0.23  2.17  3.10  2734    1
## beta2 -0.17      0 0.05 -0.26 -0.08  4119    1
## beta3 -0.47      0 0.05 -0.56 -0.37  3666    1
## sigma  0.20      0 0.01  0.17  0.22  3520    1
##
## Samples were drawn using NUTS(diag_e) at Fri May  8 13:46:23 2026.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```

The frequentist confidence intervals from `m_coarse_2` and the Bayesian 95% credible intervals are very similar for all parameters. For example, the interval for the thorax effect is (2.22, 3.15) in the frequentist model and approximately (2.20, 3.07) in the Bayesian model. Similarly, the intervals for `is_low` and `is_high` closely align across the two approaches.

The frequentist estimate of σ is 0.194, while the Bayesian posterior mean is approximately 0.20 with a 95% credible interval of (0.17, 0.22), again showing strong agreement.

This similarity indicates that the likelihood dominates the prior, meaning the data are sufficiently informative and the chosen priors are relatively weak.

The key difference lies in interpretation. A 95% confidence interval means that, under repeated sampling, 95% of such intervals would contain the true parameter. In contrast, a 95% credible interval means that, given the observed data and prior assumptions, there is a 95% posterior probability that the parameter lies within the interval.

Task 1.12

Posterior predictive intervals were calculated for new fruitflies in the **high** activity group with thorax lengths 0.7 mm and 0.9 mm. Since the model is fitted on the log-longevity scale, posterior predictive draws were generated on the log scale and then transformed back to longevity in days.

```
## # A tibble: 2 x 5
##   thorax activity posterior_median_days lower_95_PPI_days upper_95_PPI_days
##   <dbl> <chr>          <dbl>          <dbl>          <dbl>
## 1    0.7 high           28.0           19.1           41.9
## 2    0.9 high           47.7           31.8           71.4
```

For a fruitfly with thorax length 0.7 mm and high activity, the posterior median longevity is 28.1 days, with a 95% posterior predictive interval of (19.2, 42.1) days. For a thorax length of 0.9 mm, the posterior median longevity is 47.9 days, with a 95% posterior predictive interval of (32.4, 71.3) days.

The 95% posterior predictive intervals incorporate both uncertainty in the model parameters and natural variability between individual fruitflies.

The increase in posterior median longevity from 28.1 to 47.9 days as thorax length increases reflects the positive association between thorax length and longevity identified in the model.

Task 1.13

For Fruitfly 1, the thorax length is 0.8 mm and the activity group is **isolated_or_worse**. Therefore,

$$\mu_1 = \beta_0 + 0.8\beta_1.$$

For Fruitfly 2, the activity group is **high** and the thorax length is x . Therefore,

$$\mu_2 = \beta_0 + x\beta_1 + \beta_3.$$

To find the value x^* that makes the expected log-longevity equal for the two fruitflies, set $\mu_1 = \mu_2$:

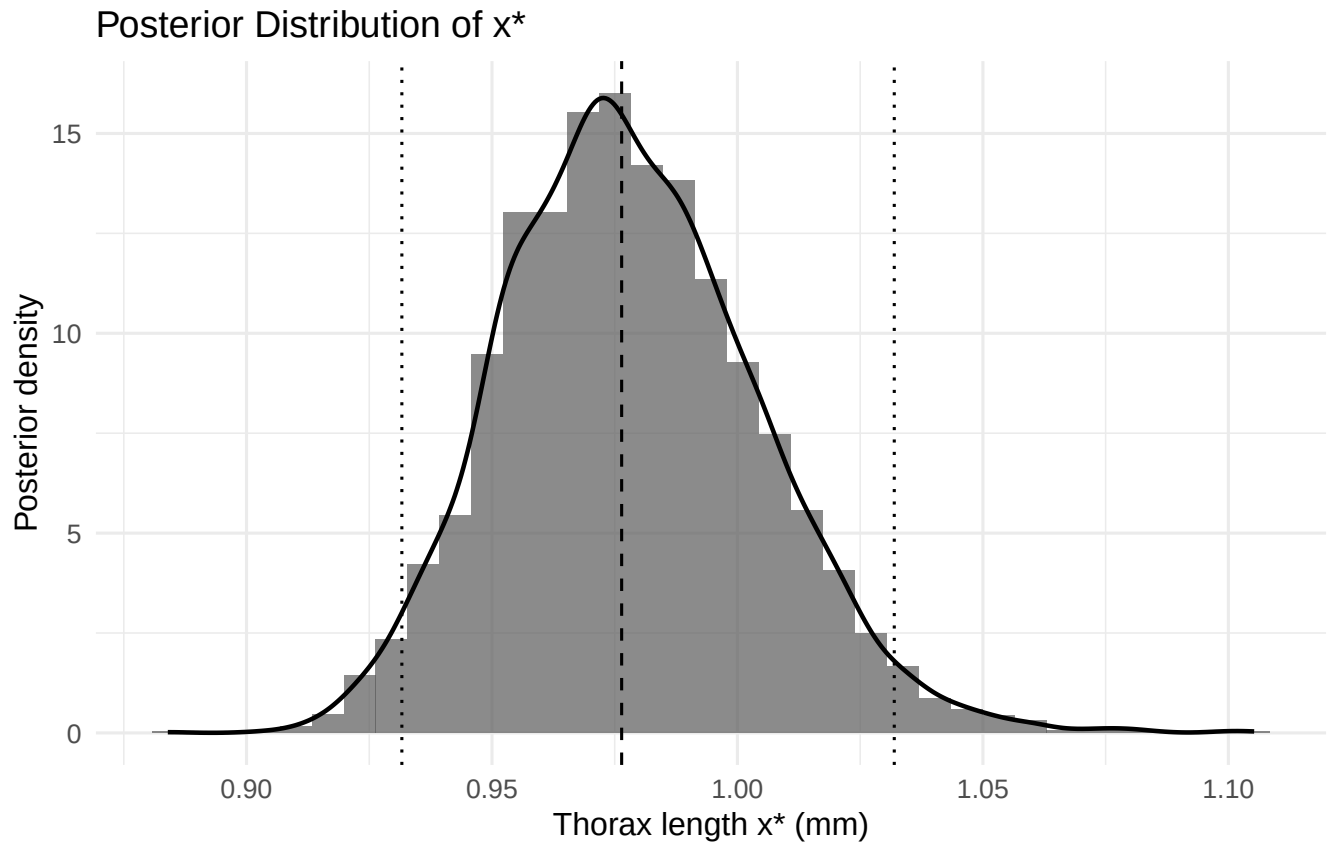
$$\beta_0 + 0.8\beta_1 = \beta_0 + x^*\beta_1 + \beta_3.$$

Rearranging gives

$$x^* = 0.8 - \frac{\beta_3}{\beta_1}.$$

Since β_1 and β_3 are uncertain parameters, the posterior distribution of x^* can be obtained by applying this formula to each posterior draw.

```
## # A tibble: 1 x 4
##   posterior_mean posterior_median lower_95_CI upper_95_CI
##   <dbl>          <dbl>          <dbl>          <dbl>
## 1    0.978         0.976         0.932         1.03
```



The posterior distribution of x^* summarises uncertainty about the thorax length required for a **high** activity fruitfly to have the same expected log-longevity as an **isolated_or_worse** fruitfly with thorax length 0.8 mm.

The posterior mean of x^* is 0.978 mm, and the posterior median is 0.977 mm. The 95% credible interval is (0.935, 1.027) mm. Therefore, conditional on the fitted Bayesian model and observed data, there is posterior probability 0.95 that the thorax length required for a **high** activity fruitfly to match the expected log-longevity of an **isolated_or_worse** fruitfly with thorax length 0.8 mm lies between 0.935 mm and 1.027 mm.

This estimate is relatively high compared with the observed thorax lengths in the data, and the upper end of the credible interval extends beyond the main observed range. Therefore, the inference should be interpreted cautiously because it partly involves extrapolation. Nevertheless, the Bayesian approach provides a direct posterior distribution for x^* , allowing uncertainty in this derived quantity to be summarised numerically and graphically.

Task 1.14

Sexual activity appears to have a measurable physiological cost for male fruitflies. After accounting for body size (thorax length), more active groups tend to have lower expected longevity than less active (**isolated_or_worse**) flies. However, the relationship between body size and longevity is consistent across groups, meaning activity affects overall lifespan rather than how size influences it. The differences between activity levels are statistically supported and biologically meaningful, suggesting that increased mating activity is associated with reduced lifespan. Overall, the evidence indicates a trade-off between reproductive activity and survival.

Task 2.1

Reading in the data, stating dimensions, and converting type into factor labelled “Y” and “N”.

```
# Reading in data
diabetes_training <- read.csv("diabetes_training.csv")
diabetes_test <- read.csv("diabetes_test.csv")
```

```
# Dimensions
dim(diabetes_training)
```

```
## [1] 250 8
```

```
dim(diabetes_test)
```

```
## [1] 282 8
```

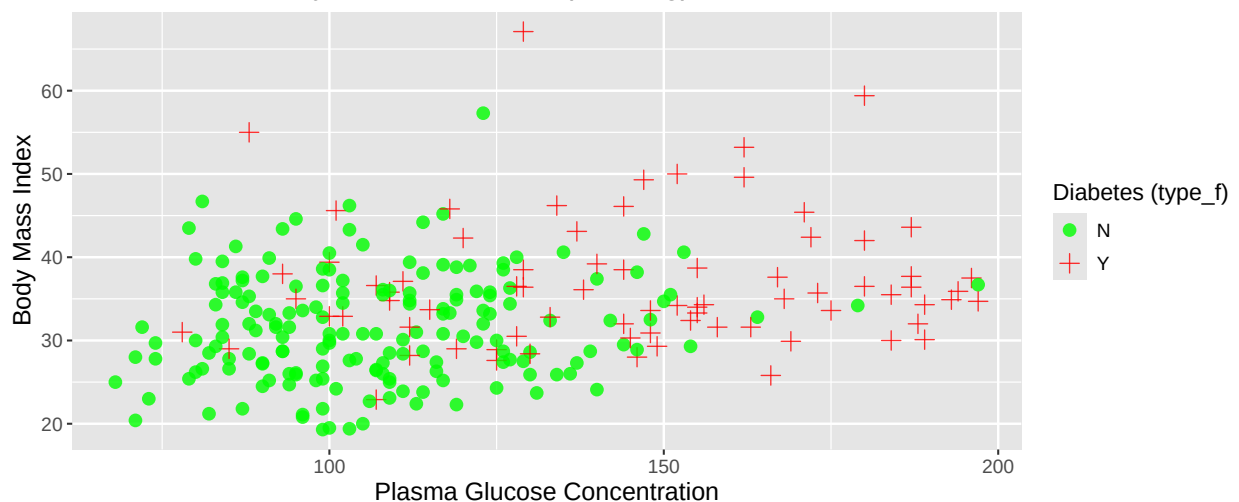
```
# Convert type to factor
diabetes_training <- diabetes_training %>%
  mutate(type_f = factor(type, levels = c("No", "Yes"), labels = c("N", "Y")))

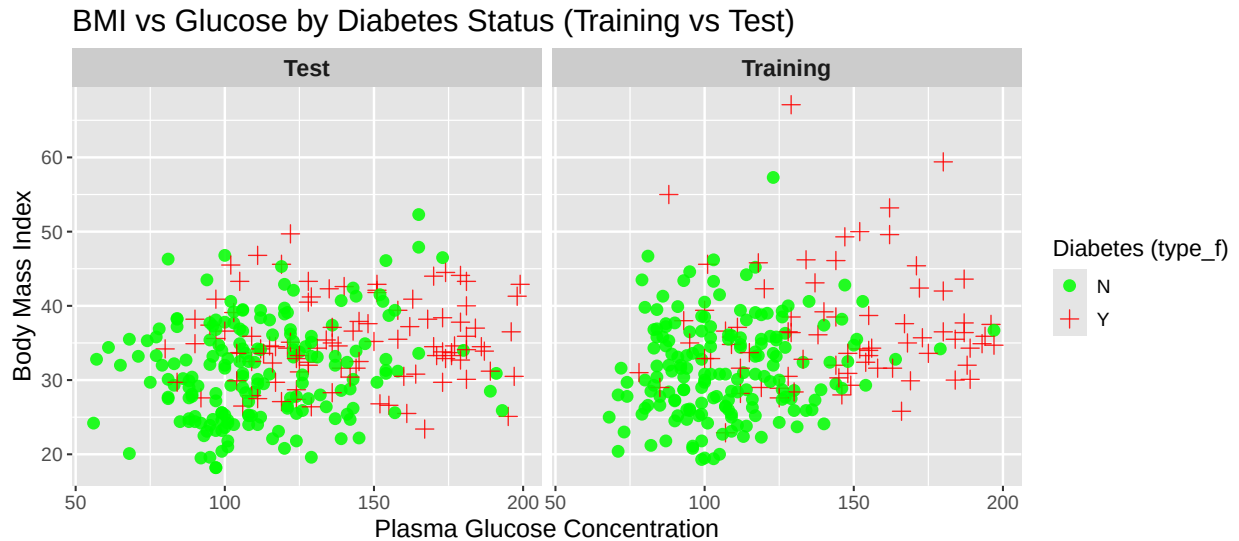
diabetes_test <- diabetes_test %>%
  mutate(type_f = factor(type, levels = c("No", "Yes"), labels = c("N", "Y")))
```

Task 2.2

First graph shows a scatter plot of bmi against glu, with points labelled by type_f (training data). Second graph shows the same for both training and test data, in a combined faceted scatter plot.

BMI vs Glucose by Diabetes Status (Training)





Task 2.3

Finding max. pregnancies in both datasets, and converting “npreg” into a factor “npreg_6_f”.

```
# Max number of pregnancies in test and training datasets
max(diabetes_training$npreg)
```

```
## [1] 14
```

```
max(diabetes_test$npreg)
```

```
## [1] 17
```

```
# Creating factor npreg_6_f (training data)
diabetes_training <- diabetes_training %>%
  mutate(npreg_6_f = fct_collapse(
    as.factor(npreg),
    "6 or more" = as.character(6:20)
  ))

# Creating factor npreg_6_f (test data)
diabetes_test <- diabetes_test %>%
  mutate(npreg_6_f = fct_collapse(
    as.factor(npreg),
    "6 or more" = as.character(6:20)
  ))

# Level ordering
diabetes_training <- diabetes_training %>%
  mutate(npreg_6_f = factor(npreg_6_f,
    levels = c("0", "1", "2", "3", "4", "5", "6 or more")
  ))
```

```
diabetes_test <- diabetes_test %>%
  mutate(npreg_6_f = factor(npreg_6_f,
                             levels = c("0", "1", "2", "3", "4", "5", "6 or more")
                             ))
```

Tabulate results

```
table(diabetes_training$npreg_6_f)
```

```
##
##      0      1      2      3      4      5 6 or more
##     40     61     32     29     19     11      58
```

```
table(diabetes_test$npreg_6_f)
```

```
##
##      0      1      2      3      4      5 6 or more
##     37     55     47     28     22     20      73
```

Task 2.4

The full binary logistic regression model was fitted before backward elimination.

```
# Fitting full model before backward elimination
model_full <- glm(type_f ~ npreg_6_f +
                  age + I(age^2) + I(age^3) +
                  glu + I(glu^2) + I(glu^3) +
                  bmi + I(bmi^2) + I(bmi^3) +
                  bp + skin + ped +
                  I(glu * bmi),
                  data = diabetes_training,
                  family = binomial)
```

The least significant variable is “npreg_6_f5”, with the highest p-value of 0.9075. As the other factors of “npreg_6_f” are not found to be statistically significant either ($p > 0.05$), the whole variable “npreg_6_f” was removed, and this was fitted as a simpler “model_1”.

The variable with the highest p-value in “model_1” is “Age^2”. However, a lower-order term cannot be removed from the model while a higher order term exists. Therefore, “Age^3” was removed as it was also statistically insignificant, and allowed for evaluation of the lower-order terms.

The least significant variable in the next model, “model_2”, is “skin” with a p-value of 0.8543. There are no hierarchical restrictions for this variable, and was therefore removed to create “model_3”.

This elimination process was repeated removing statistically insignificant variables “glu*bmi”, “glu^3”, “glu^2”, “bmi^3”, “bmi^2”, “bp” in order. The final model, “model_9”, contains five remaining variables: age, age^2, glu, bmi, and ped, all of which were significant ($p < 0.05$).

```
model_9 <- glm(type_f ~
               age + I(age^2) +
               glu +
               bmi +
               ped,
               data = diabetes_training,
               family = binomial)
```

The final model was evaluated on the test data and used to create probability values. Using a threshold of 0.5 for type ‘Y’ classification, a confusion matrix was created.

```
# Evaluation of the model on the test data
# Probability prediction
probs <- predict(model_9, diabetes_test, type = "response")

# Convert probabilities to yes or no based on 0.5 threshold
pred <- ifelse(probs > 0.5, "Y", "N")

# Create confusion matrix
cm <- table(Predicted = pred, Actual = diabetes_test$type_f)
cm
```

```
##           Actual
## Predicted   N    Y
##           N 156  47
##           Y  25  54
```

True positive/negative and false positive/negative values were used to calculate model metrics such as error, accuracy, sensitivity, specificity and precision. These metrics will help to compare performance between different models.

```
# Calculate model accuracy, sensitivity, specificity, precision, and FPR
# Extract values
TN <- cm["N","N"]
FN <- cm["N","Y"]
FP <- cm["Y","N"]
TP <- cm["Y","Y"]

# Error
error <- (FP + FN) / (TP + TN + FP + FN)

# Accuracy
accuracy <- (TP + TN) / (TP + TN + FP + FN)

# Sensitivity
sensitivity <- TP / (TP + FN)

# Specificity
specificity <- TN / (TN + FP)

# Precision
precision <- TP / (TP + FP)
```

Task 2.5

As before, the full simple binary logistic regression model was fitted before backward elimination. Backward elimination allowed for the removal of statistically insignificant variables “npreg_6_f”, “skin”, and “bp” in order. The model can be expressed mathematically as shown below:

$$Y_i \sim \text{Bernoulli}(p_i)$$

$$\log\left(\frac{p_i}{1-p_i}\right) = \beta_0 + \beta_1 age_i + \beta_2 glu_i + \beta_3 bmi_i + \beta_4 ped_i$$

As before, the final model was evaluated on the test data and used to create probability values. A confusion matrix was created using the same 0.5 threshold. The confusion matrix for this model is shown below:

```
##           Actual
## Predicted   N    Y
##           N 160  49
##           Y  21  52
```

Task 2.6

A function was created to compute the F1 score for this simpler model, from the confusion matrix.

```
# Writing a function to compute 'F1' from confusion matrix produced in 2.5
F1 <- function(cm_simple) {
  TP_simple <- cm_simple["Y","Y"]
  FP_simple <- cm_simple["Y","N"]
  FN_simple <- cm_simple["N","Y"]

  precision_simple <- TP_simple / (TP_simple + FP_simple)
  sensitivity_simple <- TP_simple / (TP_simple + FN_simple)

  f1_score <- 2 * precision_simple * sensitivity_simple / (precision_simple + sensitivity_simple)

  return(f1_score)
}
```

Task 2.7

A matrix of scatter plot for seven covariates, coloured according to “type_f” was created.

```
ggpairs(diabetes_training,
  columns = c("npreg", "age", "glu", "bmi", "bp", "skin", "ped"),
  aes(colour = type_f, alpha = 0.4),
  upper = list(continuous = "points"),
  lower = list(continuous = "smooth"),
  diag = list(continuous = "densityDiag"))
```



The plot shows that “glu” is the strongest discriminator between classes, with higher values being associated with diabetes. A weaker positive trend can be seen with variables “bmi” and “age”, while other variables show much overlap, suggesting weak predictive power.

Task 2.8

K-nearest neighbours was applied to the data. The model was trained using the training data, with 100 observations used for validation ($k=5$). A threshold of 0.5 was used to produce the confusion matrix from prediction values. Classification error rates were also calculated, along with an F1 score.

```
# Creating validation set from training data
set.seed(1)
val_index <- sample(1:nrow(diabetes_training), 100) #randomly select 100 obsv for val

validation_data <- diabetes_training[val_index, ]
training_data_knn <- diabetes_training[-val_index, ]

# Splitting X and Y (variables and type_f)
vars <- c("npreg", "age", "glu", "bmi", "bp", "skin", "ped")

train_x <- training_data_knn[, vars]
train_y <- training_data_knn$type_f

val_x <- validation_data[, vars]
val_y <- validation_data$type_f

test_x <- diabetes_test[, vars]
test_y <- diabetes_test$type_f

# Scaling using training only
train_scaled <- scale(train_x)
```

```

val_scaled <- scale(val_x,
                    center = attr(train_scaled, "scaled:center"),
                    scale = attr(train_scaled, "scaled:scale"))
test_scaled <- scale(test_x,
                     center = attr(train_scaled, "scaled:center"),
                     scale = attr(train_scaled, "scaled:scale"))

# Choosing best k value
k_values <- 1:20
val_errors <- rep(NA, length(k_values))

for (k in k_values) {
  pred_knn <- knn(train_scaled, val_scaled, cl = train_y, k = k)
  cm_knn <- table(pred_knn, val_y)

  error_knn <- (cm_knn["Y", "N"] + cm_knn["N", "Y"]) / sum(cm_knn)

  val_errors[k] <- error_knn
}

best_k <- which.min(val_errors)

# Final model on test data
pred_knn_final <- knn(train_scaled, test_scaled, cl = train_y, k = best_k)

cm_knn_final <- table(Predicted = pred_knn_final, Actual = test_y)
cm_knn_final

```

```

##           Actual
## Predicted   N   Y
##           N 166  60
##           Y   15  41

```

Gradient boosting was also applied to the data using a Bernoulli distribution for binary classifications, trained with 5000 trees, interaction depth of 4, and a shrinkage parameter of 0.1. Shrinkage value of 0.1 was selected to provide a balance between model flexibility and overfitting, allowing gradual learning while maintaining stable performance. A threshold of 0.5 was used to produce the confusion matrix from prediction values. Classification error rates were also calculated, along with an F1 score.

```

# Convert Y and N to 1 and 0
train_y_boost <- ifelse(diabetes_training$type_f == "Y", 1, 0)
test_y_boost <- ifelse(diabetes_test$type_f == "Y", 1, 0)

# Fitting boosting model
boost_model <- gbm(
  train_y_boost ~ npreg + age + glu + bmi + bp + skin + ped,
  data = diabetes_training,
  distribution = "bernoulli",
  n.trees = 5000,
  interaction.depth = 4,
  shrinkage = 0.1
)

```

```

# Predicting probabilities
probs_boost <- predict(boost_model,
                        diabetes_test,
                        n.trees = 5000,
                        type = "response")

# Converting probabilities back into Y and N classes
pred_boost <- ifelse(probs_boost > 0.5, "Y", "N")

# Creating confusion matrix
cm_boost <- table(Predicted = pred_boost,
                  Actual = diabetes_test$type_f)

cm_boost

```

```

##           Actual
## Predicted   N   Y
##           N 152  52
##           Y  29  49

```

A pruned tree model was applied to the data. A threshold 0.5 was used for classification from prediction values. Performance metrics and an F1 score was produced. The tree plot and confusion matrix are as seen below:

```

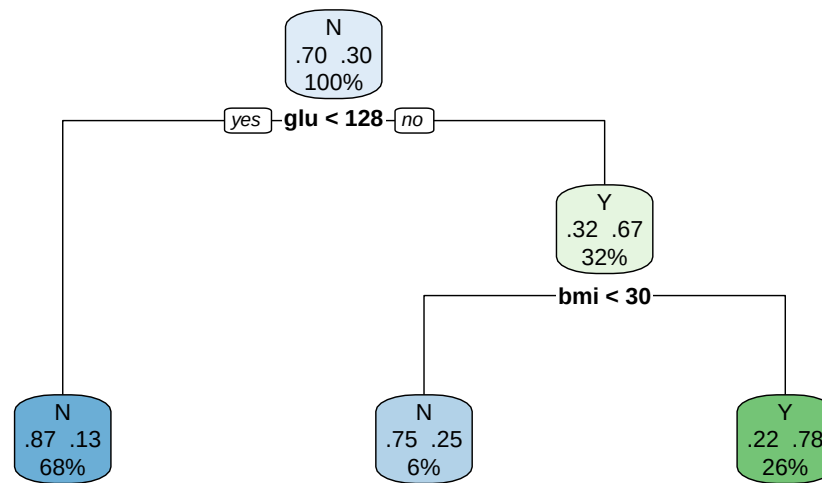
# Fitting full tree
tree_model <- rpart(
  type_f ~ npreg + age + glu + bmi + bp + skin + ped,
  data = diabetes_training,
  method = "class"
)

# Find optimal CP
best_cp <- tree_model$cptable[
  which.min(tree_model$cptable[, "xerror"]),
  "CP"
]

# Pruning the tree
pruned_tree <- prune(tree_model, cp = best_cp)

# Plot the tree
rpart.plot(pruned_tree, type = 2, extra = 104)

```

```

# Predictions on test data
tree_pred <- predict(pruned_tree,
                     diabetes_test,
                     type = "class")

# Confusion matrix
cm_tree <- table(Predicted = tree_pred,
                 Actual = diabetes_test$type_f)

cm_tree

```

```

##           Actual
## Predicted   N    Y
##           N 151  44
##           Y  30  57

```

Random forest model with 500 trees was applied to the data. A threshold 0.5 was used for classification from prediction values. Performance metrics and an F1 score was produced.

```

# Fitting initial model
rf_model <- randomForest(
  type_f ~ npreg + age + glu + bmi + bp + skin + ped,
  data = diabetes_training,
  ntree = 500,
  importance = TRUE
)

# Tuning mtry as default may not be optional
set.seed(1)

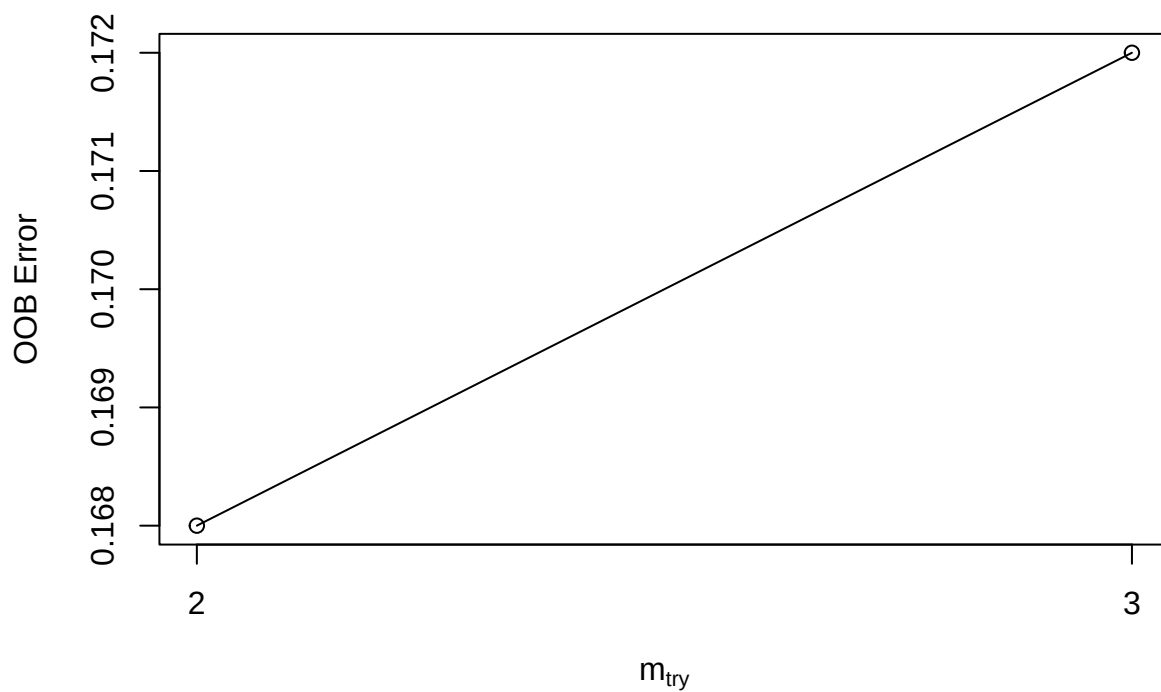
mtrytune <- tuneRF(
  diabetes_training[, c("npreg", "age", "glu", "bmi", "bp", "skin", "ped")],
  diabetes_training$type_f,
  stepFactor = 1.5,
  improve = 0.01,

```

```

ntreeTry = 500
)

```



```

# Fit final tuned model
best_mtry <- mtrytune[which.min(mtrytune[,2]), 1]

rf_final <- randomForest(
  type_f ~ npreg + age + glu + bmi + bp + skin + ped,
  data = diabetes_training,
  ntree = 500,
  mtry = best_mtry,
  importance = TRUE
)

# Predictions on test data
rf_pred <- predict(rf_final, diabetes_test)

# Confusion matrix
cm_rf <- table(Predicted = rf_pred,
               Actual = diabetes_test$type_f)

cm_rf

```

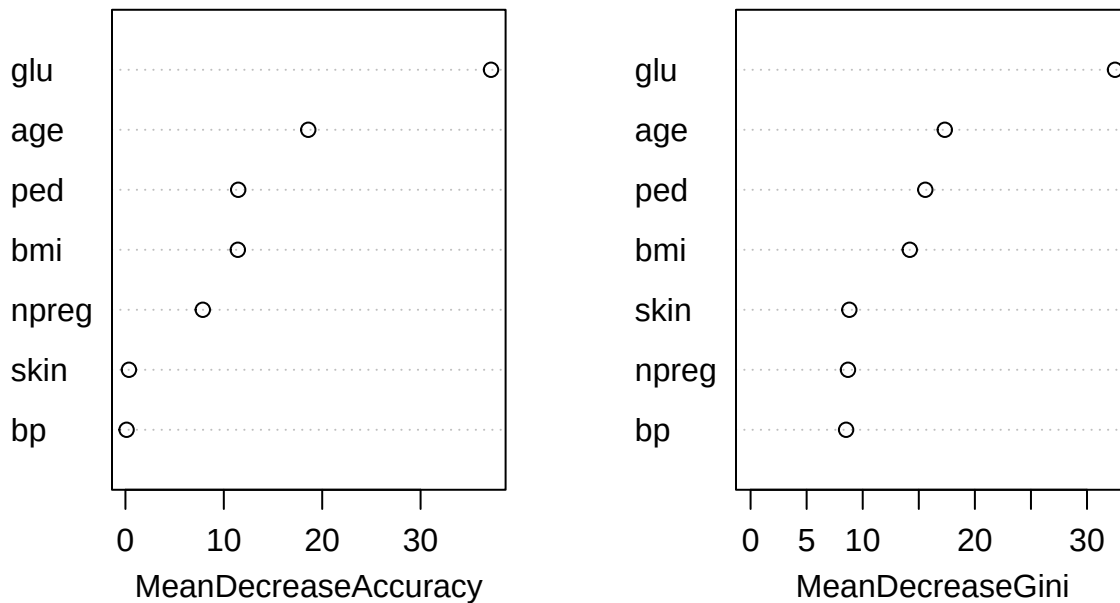
```

##           Actual
## Predicted   N   Y

```

```
##          N 154  48
##          Y  27  53
```

rf_final



ROC curves and Area Under Curve (AUC) measures were produced for simple binary logistic regression and gradient boosting models as shown below. The code for the boosting model graph has been omitted due to similarity.

```
# ROC curve for simple binary logistic regression model
# Creating prediction object
pred_simple_obj <- prediction(
  predictions = probs_simple,
  labels = diabetes_test$type_f
)

# Computing ROC curve
roc_simple <- performance(
  pred_simple_obj,
  measure = "tpr",
  x.measure = "fpr"
)

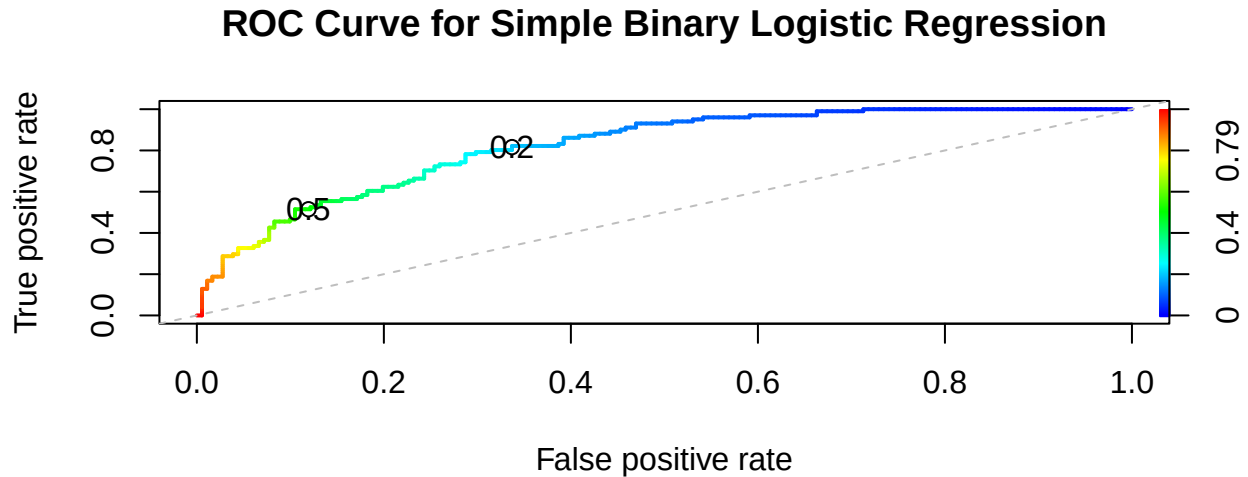
# Plotting ROC curve
plot(
  roc_simple,
  colorkey = TRUE,
  colorize = TRUE,
  lwd = 2,
```

```

main = "ROC Curve for Simple Binary Logistic Regression",
print.cutoffs.at = c(0.2, 0.5),
cex.axis = 0.9,
cex.lab = 1
)

abline(a = 0, b = 1, lty = 2, col = "grey")

```

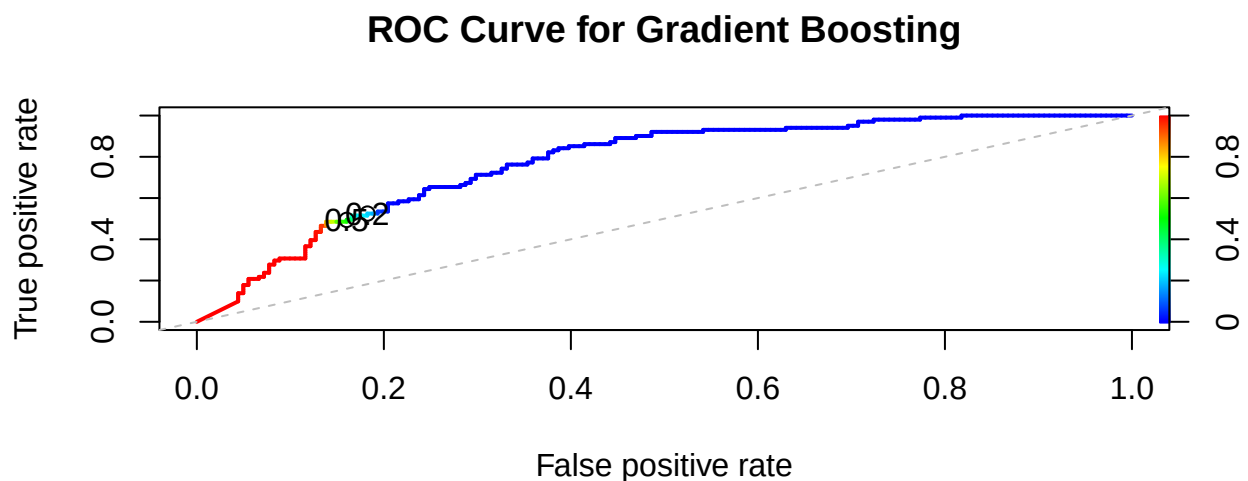


```

# Computing AUC
auc_simple <- performance(pred_simple_obj, measure = "auc")
auc_simple_value <- auc_simple@y.values[[1]]
auc_simple_value

```

```
## [1] 0.8194847
```



[1] 0.7733166

Table 1: Performance metrics for machine learning models

Model	Error	Accuracy	Sensitivity	Specificity	Precision	F1
Logistic Regression	0.2482	0.7518	0.5149	0.8840	0.7123	0.5977
KNN	0.2600	0.7340	0.4059	0.9171	0.7321	0.5223
Boosting	0.2872	0.7128	0.4851	0.8398	0.6282	0.5475
Decision Tree	0.2624	0.7376	0.5644	0.8343	0.6552	0.6064
Random Forest	0.2660	0.7340	0.5248	0.8508	0.6625	0.5856

Task 2.9

Multiple machine learning methods were applied to the diabetes data, and compared using classification error, accuracy, sensitivity, specificity, precision and F1 score. Overall, model performance was broadly similar but there were important differences in their ability to correctly identify diabetes cases.

The simple binary logistic regression model achieved the highest accuracy (75.18%), the lowest error rate (24.82%), and the highest AUC value of 0.8195. This indicates that the model was effective at distinguishing between diabetic and non-diabetic individuals. The model also achieved a relatively high precision of 71.23%, meaning that a large proportion of the predicted diabetic individuals were correctly classified. This model was also easily interpretable, with plasma glucose concentration, body mass index, age, and diabetes pedigree function being identified as significant predictors of diabetes.

The decision tree model achieved the highest sensitivity of 56.44% and the highest F1 score of 0.6064, meaning that it identified the largest proportion of true diabetic cases while maintaining a balance between false negatives and false positives. Precision (65.52%) was slightly worse than logistic regression and K-nearest neighbours, meaning that there were a greater number of false positive classifications. However, the tree structure was easily interpretable, with glucose concentration and BMI being deemed the most important decision rules.

KNN achieved the highest specificity of 91.71% as well as the highest precision at 73.21%, indicating that when the model predicted diabetes, it was most likely to be correct. However, the model also had the lowest sensitivity, achieving only 40.59%, meaning that many true diabetes cases were missed.

Random forest showed balanced performance in comparison, achieving moderate sensitivity, specificity, and precision values. However, it did not outperform the simpler binary logistic regression model overall. The variable importance measures from the random forest model also showed that glucose concentration was the strongest predictor diabetes, consistent with other models.

Gradient boosting, in this scenario, did not perform as well as the other models. Although boosting is a highly flexible machine learning technique, it produced the lowest accuracy, lowest precision, and highest error, as well as the second lowest sensitivity, specificity, and F1 score. This may be due to the structure of the diabetes data being relatively simple, and better interpreted by simpler models like logistic regression and decision tree.

In conclusion, the simple binary logistic regression model would be best suited for this application. It achieved the strongest performance across various metrics, while remaining highly interpretable. This is an important consideration in a medical setting, as clinicians must understand how predictions are being made. The decision tree model could be considered to be a strong alternative, due to its similar performance in metrics, as well as the easy-to-understand graphical illustration.